コンソーシアム型ブロックチェーンにおける デプロイ経路の限定と監査強制による スマートコントラクトのライフサイクル管理

山本 和弥[1,a)]　瀧田 愼[1]　野村 健太[2]　林田 淳一郎[2]　石井 龍[2]　佐古 健太郎[2]　永井 達也[2]
渡邉 卓弥[2]　齋藤 恆和[2]　高田 雄太[2]　熊谷 裕志[2]　神薗 雅紀[2,3]　葛野 弘樹[1]　白石 善明[1,b)]

概要：ブロックチェーン上にデプロイされたスマートコントラクトは不変性を持ち，脆弱性の修正や仕様変更が困難である．アップグレードや停止を可能にする手法は提案されているが，個々のコントラクトを対象とするため，チェーン上のコントラクト全体を管理対象にできない．本提案では，コンソーシアム型ブロックチェーンを対象とし，監査，デプロイ，アップグレード，停止からなるライフサイクルをレジストリコントラクトで一元的に管理する．加えて，基盤の権限管理によりネットワーク構築後のデプロイ経路をレジストリ経由に限定し，管理者と基盤設定を信頼する前提の下で，管理外コントラクトの追加を防ぐ．実装したシステムに対する静的解析の指摘には，脅威モデルの下で 4 つの守るべき性質を破る経路がないことを確認した．また，権限のないアカウントからのデプロイはノードが，管理者以外による操作はレジストリのアクセス制御が拒否することを確認した．

キーワード：ブロックチェーン，スマートコントラクト，ライフサイクル管理，アクセス制御

Smart Contract Lifecycle Management by Restricting Deployment Paths and Enforcing Audits on a Consortium Blockchain

Kazuya Yamamoto[1,a)]　Makoto Takita[1]　Kenta Nomura[2]　Junichiro Hayata[2]　Ryu Ishii[2]
Kentaro Sako[2]　Tatsuya Nagai[2]　Takuya Watanabe[2]　Tsunekazu Saito[2]　Yuta Takata[2]
Hiroshi Kumagai[2]　Masaki Kamizono[2,3]　Hiroki Kuzuno[1]　Yoshiaki Shiraishi[1,b)]

Abstract: Smart contracts deployed on a blockchain are immutable, which makes vulnerability fixes and specification changes difficult. Existing mechanisms enabling upgrade or suspension target individual contracts and thus cannot bring all contracts on the chain under management. We propose a management system for consortium blockchains in which a registry contract centrally manages a lifecycle consisting of audit, deployment, upgrade, and suspension. In addition, after network construction, the platform's permissioning restricts deployment to the path through the registry, preventing the addition of unmanaged contracts under the assumption that the administrator and the platform configuration are trusted. For the implemented system, we confirmed that none of the static-analysis findings yields a path that violates the four required properties under our threat model. We also confirmed that the node rejects deployments from unauthorized accounts and that the registry's access control rejects operations by non-administrators.

Keywords: blockchain, smart contract, lifecycle management, access control

[1] 神戸大学
Kobe University
[2] デロイト トーマツ サイバー合同会社
Deloitte Tohmatsu Cyber LLC
[3] 一般社団法人デロイト トーマツ戦略研究所
Deloitte Tohmatsu Strategy Institute
[a)] yamamoto.kazuya@gsuite.kobe-u.ac.jp
[b)] zenmei@port.kobe-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

ブロックチェーンは, 改ざん耐性を備えた台帳を複数の参加者間で共有する分散型台帳技術である [1]．Ethereum [2] に代表されるブロックチェーン基盤では，スマートコントラクト（以下，コントラクト）として記述されたプログラムをブロックチェーン上で実行できる．このようなコントラクトの実行機能は，参加者を限定したコンソーシアム型ブロックチェーンにおいても利用されており，複数の主体間で情報や処理を共有する基盤として，様々なアプリケーションへの応用が検討されている [3]．

ブロックチェーン上にデプロイされたコントラクトは，第三者による改ざんが困難であり，コードの同一性が保たれるという不変性を持つ．一方で，デプロイ後に発見された脆弱性の修正や，運用中に生じた仕様変更への対応を困難にする [4]．このように，不変性はセキュリティ上の利点であると同時に，保守性を損なう要因でもある．

2016 年に発生した The DAO 事件 [5] は，デプロイ後のコントラクトを容易に修正できないことが問題となった代表的な事例である．The DAO のコントラクトには再入可能性の脆弱性が存在し，脆弱な処理を即座に停止する機構や修正版へ差し替える機構が備わっていなかった．その結果，6,000 万ドル相当（2016 年当時）の暗号資産が不正に引き出され，最終的に，ブロックチェーン全体のハードフォークによる対応に至った．

このような問題は，コンソーシアム型ブロックチェーンを利用したシステムにおいても考慮する必要がある．脆弱性の修正や機能追加に対応するためには，デプロイ後のコントラクトを継続的に管理する仕組みが必要となる．管理の手段として，プロキシパターンをはじめとするアップグレード可能なコントラクトの構造が用いられている [4]．一方，こうしたアップグレード機構は，個々のコントラクトを対象としており，ネットワーク全体の管理を目的としたものではない．

本提案では，Ethereum の実行環境である EVM (Ethereum Virtual Machine) と互換性を持つコンソーシアム型ブロックチェーンを対象とし，デプロイ前にコントラクトの安全性を確認する監査，デプロイ，アップグレード，停止からなるライフサイクルをチェーン上で一元的に管理する．管理対象となるコントラクトの情報と状態をレジストリコントラクト（以下，レジストリ）に記録し，その状態遷移はレジストリを介して制御する．加えて，ブロックチェーン基盤の権限管理機能を用いて，コントラクトのデプロイ経路をレジストリ経由に限定する．本提案の貢献は次の 3 点である．

ネットワーク全体を対象とするデプロイ経路の管理： ブロックチェーン基盤の権限管理により，アカウントによるコントラクトの直接デプロイを禁止し，デプロイ経路はアクセス制御を備えたレジストリ経由に限定する．デプロイ経路の限定により，3 章に示す脅威モデルの下で，ネットワーク構築時に配置されるコントラクトを除き，管理外のコントラクトが新たにデプロイされることを防ぐ．

スマートコントラクトのライフサイクル管理： 監査，デプロイ，アップグレード，停止からなるコントラクトのライフサイクルをレジストリで管理し，各状態への遷移を管理者の承認に基づいて制御する．

GoQuorum 上での実装と実現可能性の提示： 提案手法をコンソーシアム型ブロックチェーン基盤 GoQuorum [6] 上に実装し，提案するライフサイクル管理機構が EVM 互換のコンソーシアム型ブロックチェーン上で実現可能であることを示す．

2. 関連研究

本章では，コントラクトのアップグレードや停止を実現する要素技術，これらを管理する既存方式，およびライフサイクル管理機能を備えるブロックチェーン基盤について順に述べる．

デプロイ後のコントラクトを変更可能にする代表的な手法として，Ethereum ではプロキシパターン (Proxy Pattern) がある [4]．プロキシパターンでは，ストレージを保持するプロキシコントラクトと，処理を実装するロジックコントラクトを分離する．コントラクト実行時は，プロキシコントラクトが `delegatecall` と呼ばれる低レベル呼び出しによって処理をロジックコントラクトに委譲する構造をとる．プロキシが参照するロジックコントラクトを変更することで，アドレスとストレージを維持したまま実装を差し替えることができる．UUPS (Universal Upgradeable Proxy Standard) [7] をはじめ，プロキシパターンをもとにした複数のアップグレード手法が提案されている．

Wöhrer ら [8] は，Ethereum 上のコントラクトに用いられるセキュリティパターンを整理し，その 1 つとして緊急停止 (Emergency Stop) を挙げている．コントラクト内部で稼働状態を管理し，権限を持つアカウントが状態を切り替えることで，対象となる関数の実行を停止する方式である．停止状態を解除することで，対象の関数を再び実行可能にできる．

以上のパターンは，アップグレードや停止を実現する仕組みを提供する一方，これらの機構を採用するか，またどのような手続きで操作するかは，個々のコントラクトの設計や運用に委ねられる．Qasse ら [9] は，Ethereum 上にデプロイされた 4,400 万件を超える Solidity コントラクトを調査し，アップグレード可能な特徴を持つものは約 3%であり，そのうち実際にアップグレードが行われたコントラクトは 0.34%であったと報告している．アップグレードを組織的に運用するには，その実現機構に加えて，誰がどの

© 2026 Information Processing Society of Japan

ような手続きで実行を承認するかを定めるガバナンスが必要となる．

TRUST [10] は，コントラクトのアップグレードに，複数の主体による承認と追跡可能な記録を導入したライフサイクル管理アーキテクチャを提案し，実装している．Registry と呼ばれるチェーン上のコントラクトが，管理対象コントラクトの状態，バージョン履歴，承認手続きを管理する．アップグレードは所定の承認条件を満たした場合に実行され，バージョン履歴に基づいて過去のバージョンへ戻すこともできる．

一方，TRUST はブロックチェーン基盤自体を変更せず，チェーン上のスマートコントラクトによってライフサイクル管理を実現する．そのため，ブロックチェーン基盤におけるコントラクトのデプロイ経路自体は制限されず，TRUST を経由せずにデプロイされたコントラクトは管理対象外となる．

Wang ら [11] は，EVM 互換のプライベート型ブロックチェーンを対象に，コントラクトのデプロイを制御する仕組みを提案している．ノードの手前にリバースプロキシ (Reverse Proxy) を配置し，コントラクトのデプロイを行うトランザクションを検出する．トランザクションの送信元のアカウントを許可リスト (whitelist) と照合し，登録されている場合はノードへ転送し，登録されていない場合は拒否する．これにより，許可されたアカウントにコントラクトのデプロイを制限する．ただし，デプロイの可否は送信元アカウントに基づいて判定され，デプロイされるコントラクトのコード内容は判定対象としない．また，判定はノードの外部で行われるため，実運用ではリバースプロキシを迂回したアクセスを防ぐために，ノードの RPC を別途保護する必要がある．

ここまでに述べた手法は，既存のブロックチェーン基盤上に，追加の機構を設けることで，コントラクトのアップグレードや停止，デプロイを制御するものである．Hyperledger Fabric [12] は，トランザクションの実行と順序付けを分離したコンソーシアム型ブロックチェーン基盤である．バージョン 2.0 以降，スマートコントラクトに相当するチェーンコードのライフサイクル管理を基盤の機能として備えている．チェーンコードは，参加組織による承認を含む所定の手続きを経て，ネットワーク上で利用可能となる．一方，Fabric は EVM とは異なる実行環境を採用しているため，EVM 互換のブロックチェーン基盤にその管理機構を適用することはできない．

関連技術および既存方式と本提案の比較を表 1 に示す．本提案の特徴は，EVM 互換のブロックチェーン基盤において，デプロイ経路をレジストリ経由に限定する点にある．これにより，管理外コントラクトの追加を防止し，ネットワーク上のコントラクトをライフサイクル管理の対象とする．その上で，既存のアップグレードおよび停止機構をライフサイクルに組み込み，レジストリを通じて一元的に管理する．

表 1 関連手法と本提案の比較．(1) アップグレードの手段，(2) 停止の手段，(3) 承認権限の分散，(4) デプロイの制御，(5) 管理外コントラクトの追加防止，(6)EVM 互換

	(1)	(2)	(3)	(4)	(5)	(6)
Proxy Pattern [4]	✓	–	–	–	–	✓
Emergency Stop [8]	–	✓	–	–	–	✓
TRUST [10]	✓	–	✓	–	–	✓
Reverse Proxy [11]	–	–	–	✓	–	✓
Fabric [12]	✓	–	✓	✓	✓	–
本提案	✓	✓	–	✓	✓	✓

3. 脅威モデルと要件

提案するシステムは，複数組織が参加するコンソーシアム型ブロックチェーンを対象とする．参加組織がそれぞれノードを運用する構成とし，ネットワーク設定は構築時に代表組織が行い，他の参加組織は変更権限を持たない．ブロックチェーン基盤は，アカウント単位でトランザクション（以下，Tx）の送信可否およびコントラクトのデプロイ可否を制御できる機能を備えることを前提とする．

信頼前提：代表組織の管理者は，秘密鍵を適切に管理し，監査結果に基づくデプロイ，アップグレード，および停止の判断を，正当な手続きに従って行うものとする．ネットワーク構築時に配置されるレジストリは，想定した仕様に従う正当な実装であるものとする．また，ノードが Tx の受理判定に用いる権限管理用のコントラクトおよび権限設定は正しく構成されているものとする．コンセンサス機構，バリデータ，およびノード間通信についても，正常に機能するものとする．

攻撃者の能力と目標：想定する攻撃者は，管理者以外の参加者アカウントおよびネットワークに登録されていないアカウントである．参加者アカウントは Tx 送信権限を持ち，チェーン上のコントラクトを呼び出すことができる．一方，登録されていないアカウントから送信された Tx は受理されない．攻撃者は管理者の秘密鍵を保持せず，管理者になりすますことはできないものとする．攻撃者の目標は，レジストリを経由せず，監査・承認を受けていないコントラクトを生成すること，および管理者の承認を経ない管理対象のコントラクトの状態変更である．

管理者による意図的な不正操作，管理者鍵の紛失または漏洩，ブロックチェーン基盤自体の脆弱性，ネットワーク設定の不正な改変，ならびに権限管理機構およびコンセンサス機構の侵害は，本評価の脅威モデルに含めない．

守るべき性質：

性質 1：管理外コントラクトの追加防止　ネットワーク構築時に配置されるレジストリおよび権限管理用コントラクトを除き，新たなコントラクトの生成はレジスト

© 2026 Information Processing Society of Japan

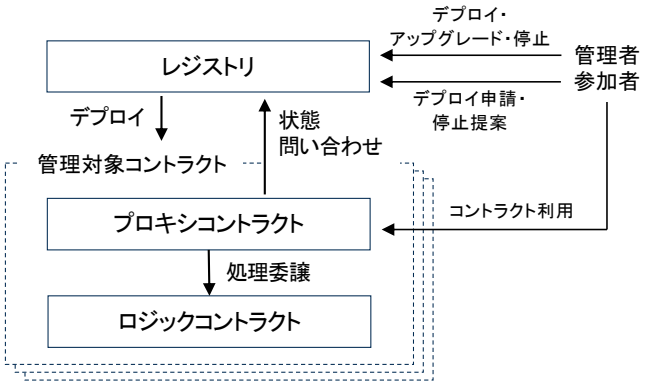


図 1 レジストリによるコントラクト管理の構成と操作

リを経由する場合に限られる．これにより，チェーン上に追加されるコントラクトは，管理者の監査・承認を経てレジストリの管理下に置かれる．

性質 2：ライフサイクル操作の権限制御 管理対象コントラクトのデプロイ，アップグレード，および停止は，管理者のみがレジストリを通じて実行できる．

性質 3：停止の有効性 停止中は，プロキシコントラクトへのすべての呼び出しが，ロジックコントラクトへの委譲前に遮断される．

性質 4：アップグレード経路の限定 プロキシコントラクトが参照するロジックコントラクトの変更は，レジストリを経由する場合に限られる．

以上の 4 つの性質を満たすため，ブロックチェーン基盤の権限設定とライフサイクル管理に必要なコントラクトからなるシステムを設計する．

4. 提案手法

4.1 全体アーキテクチャ

レジストリによるコントラクト管理の構成と操作を図 1 に示す．本提案では，レジストリコントラクト，プロキシコントラクト，およびロジックコントラクトの 3 種類を用いる．レジストリは，ネットワーク構築時にデプロイされる．管理対象のコントラクトには，アップグレードおよび停止を可能とするためプロキシパターンを採用し，プロキシコントラクトとロジックコントラクトの対を構成単位とする．レジストリはネットワークに 1 つのみ配置し，複数の対を一元的に管理する．

管理者と参加者はレジストリを介して，管理対象コントラクトのライフサイクルに関する操作を行う．参加者は，新規デプロイおよびアップグレードの申請と，停止提案の登録を行う．管理者は，申請内容を監査し，承認したコントラクトのデプロイ，アップグレード，および停止を実行する．一方，管理対象コントラクトの機能を利用する際は，レジストリを経由せずプロキシコントラクトを直接呼び出す．

4.2 デプロイ経路の限定

管理外のコントラクトがチェーン上にデプロイされることを防ぐため，デプロイ経路をレジストリに限定する．ブロックチェーン基盤に必要な機能は，デプロイ権限を持たないアカウントからのデプロイ Tx をノードが受理しないことである．ネットワーク構築時にレジストリをデプロイした後，管理者アカウントを含むすべてのアカウントからデプロイ権限を剥奪する．これにより，アカウントから直接送信されたデプロイ Tx はノードによって拒否される．ただし，デプロイ Tx かどうかの判定はアカウントからの送信時にのみ行われ，コントラクト内部からの生成はこの権限制御の対象外である．そのため，レジストリは管理対象のコントラクトを生成できる．

ここで，レジストリ以外のコントラクトが生成機能を持つ場合，監査を経ないコントラクトを追加する経路となりうる．そこで管理者の監査は，対象のコントラクトが CREATE および CREATE2 [13] による生成機能を持たないことの確認を含むものとする．EVM においてコントラクト生成に用いられる命令はこの 2 つであり，確認の範囲は通常の関数に加え，コンストラクタおよび `delegatecall` 先のコードにも及ぶ．

以上より，3 章の脅威モデルの下で，基盤の権限管理によりアカウントからの直接デプロイを防ぎ，監査によってレジストリ以外からの生成経路を排除する．さらに，レジストリのデプロイ関数を管理者のみに制限することで，コントラクトの生成経路をレジストリに限定する．

4.3 スマートコントラクトのライフサイクル

基盤の権限管理が保証するのはレジストリを介さないコントラクトのデプロイを防ぐことであり，デプロイされるコントラクトが監査済みであることまでは保証しない．そこで，唯一のデプロイ経路であるレジストリに監査・承認の手続きを組み込み，監査・承認を経た場合のみデプロイへ遷移できるライフサイクルを設ける．

管理対象のコントラクトは，監査中，稼働中，停止判断中，および停止中の 4 状態をとる．監査で棄却された登録は，その時点でライフサイクルを終了する．各コントラクトの状態遷移に関する記録はすべてチェーン上に残る．スマートコントラクトのライフサイクルを図 2 に示す．

デプロイ：新たにコントラクトがデプロイされるまでの手順は次のとおりである．

(1) 参加者は，作成したコントラクトのコントラクト名，説明，およびソースコードの keccak256 ハッシュ値をレジストリに登録し，デプロイを申請する．登録されたコントラクトは監査中となる．ソースコードのファイル自体は，チェーン外で管理者へ渡す．

(2) 管理者は，受け取ったソースコードのハッシュ値とレジストリに登録された値を照合し，静的解析等により

© 2026 Information Processing Society of Japan

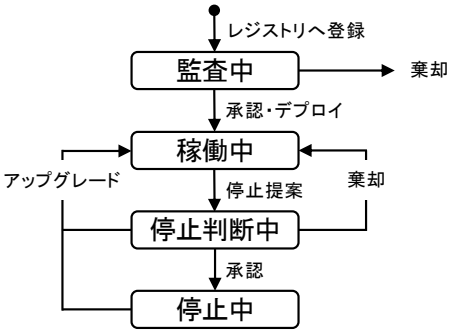


図 2 スマートコントラクトのライフサイクル

ソースコードを監査する．

(3) 承認する場合，管理者は監査したソースコードを自身でコンパイルし，生成したデプロイ用のバイトコードを添えてレジストリのデプロイ関数を呼び出す．棄却する場合はデプロイを行わず，棄却したことをレジストリに記録する．

(4) レジストリは，デプロイ関数に渡されたバイトコードを用いて，プロキシコントラクトおよびロジックコントラクトをデプロイする．その後，デプロイ用バイトコードの keccak256 ハッシュ値を記録して，状態を稼働中とする．

レジストリは，申請時のソースコードのハッシュ値とデプロイに用いたバイトコードのハッシュ値を保持する．監査対象のソースコードを同一のビルド条件で再コンパイルし，得られたデプロイ用バイトコードのハッシュ値をレジストリの記録と照合することで，デプロイに用いられたバイトコードが監査を受けたソースコードから生成されたことを検証可能とする．

アップグレード：アップグレードは，新規デプロイ時と同様に申請，監査，および承認の手続きを経て行う．承認後，レジストリは新しいロジックコントラクトをデプロイし，既存のプロキシコントラクトに参照先の変更を指示する．

プロキシコントラクトはレジストリからの変更指示のみを受理し，レジストリのアップグレード操作は管理者のみに許可する．これにより，管理者の承認を経ないロジックコントラクトの差し替えを防ぐ．

停止：コントラクトの停止が必要となった際，参加者が対象のコントラクトと停止の提案理由をレジストリに登録する．提案が登録されたコントラクトは停止判断中となるが，機能は維持される．管理者が提案を承認すると停止中へ遷移し，棄却されると稼働中へ復帰する．また，停止中のコントラクトは，単独の停止解除操作では復帰せず，監査・承認を経たアップグレードによって稼働中へ復帰する．

プロキシコントラクトは，呼び出しを受けるたびにレジストリへ対象コントラクトの状態を問い合わせる．停止中の場合はロジックコントラクトへ処理を委譲せず，呼び出しを遮断する．状態の判定を委譲前に行うため，読み取り専用の呼び出しを含むすべての呼び出しが遮断される．

ロジックコントラクトのアドレスを直接呼び出した場合は，プロキシコントラクトを経由しないため停止の判定を受けない．ただし，直接の呼び出しが読み書きするのはロジックコントラクト自身のストレージであり，管理対象のコントラクトの状態を保持するプロキシコントラクトのストレージは変更されない．停止の対象となるのは，プロキシコントラクトを経由した呼び出しに限られる．

5. 実装

5.1 実装環境とネットワーク構成

本稿では，GoQuorum 23.4.0 によるテストネットワーク上に提案システムを構築した．ネットワークは 4 台のバリデータノードと 1 台の RPC ノードで構成し，コンセンサス機構には QBFT を用いた．ブロック生成間隔は 5 秒，ガス価格は 0 とした．

コントラクトは Solidity で記述し，コンパイラにはバージョン 0.8.24 を用いた．コンパイル時の最適化を有効にし，runs を 200 に設定した．参加者と管理者が操作するフロントエンドは React および TypeScript で実装し，ブロックチェーンとの通信には ethers.js を，Tx の署名には MetaMask を用いた．

ブロックチェーンネットワークの権限管理には，GoQuorum が持つ Enhanced Network Permissioning [14] 機能を用いた．Enhanced Network Permissioning では，アカウントに割り当てる役割によって，Tx の送信やコントラクトのデプロイに関する権限を制御できる．ノードは，Tx の受理可否を判定する際に，ネットワーク構築時に配置される権限管理用のコントラクト群を参照する．

本実装では，`permissionModel` に `v2` を指定し，権限を次のように設定した．ネットワーク構築時は管理者のアカウントのみにデプロイ権限を与え，管理者がレジストリをデプロイする．レジストリのデプロイ後は管理者からもデプロイ権限を剥奪し，デプロイ権限を持つアカウントが存在しない状態とする．参加者には，Tx の送信を許可する一方，コントラクトのデプロイは許可しない．また，ネットワークに登録されていないアカウントからの Tx は受理されない．

5.2 コントラクトとフロントエンドの実装

4 章で述べた 3 種類のコントラクトを実装した．レジストリは，管理対象のコントラクトごとに，コントラクト名，説明，プロキシコントラクトのアドレス，状態，バージョン等の情報，申請者のアドレス，申請時刻，およびソースコードとデプロイ用バイトコードのハッシュ値を保持する．デプロイ，アップグレード，および停止を実行する関数には，レジストリが保持する管理者アドレスと呼び出し元アドレスを照合するアクセス制御を設けた．

© 2026 Information Processing Society of Japan

プロキシコントラクトは，アップグレードと停止に関わる関数を持つ．プロキシコントラクトの実装は，すべての管理対象のコントラクトで共通とする．ロジックコントラクトのアドレスと，状態の問い合わせ先となるレジストリのアドレスは，ERC-1967 [15] が定めるストレージスロットに保持し，ロジックコントラクトの通常のストレージとの衝突を避ける．また，プロキシコントラクトは状態の問い合わせ時に，呼び出し元のアカウントをレジストリへ記録する．レジストリは管理対象のコントラクトごとに，呼び出し回数と呼び出し元の記録を保持する．この利用記録は 3 章の 4 つの性質には関与せず，運用時に利用状況を把握するための付加機能である．

参加者は，初期化関数を備えた基底コントラクトを継承してロジックコントラクトを実装する．管理者は，アップグレード前後のロジックコントラクト間におけるストレージレイアウトの互換性に加え，CREATE および CREATE2 による生成機能が存在しないことを確認する．この確認は，通常の関数だけでなく，コンストラクタおよび `delegatecall` 先のコードも対象とする．

レジストリを操作するため，Web フロントエンドを実装した．参加者は，各コントラクトの詳細情報をフロントエンドで参照する．デプロイ申請時は，コントラクト名，説明，およびソースコードを指定する．フロントエンドはソースコードの keccak256 ハッシュ値を算出し，これらの情報を含むデプロイ申請の Tx を作成する．停止提案時は，対象のコントラクトと提案理由を指定して Tx を作成する．Tx は MetaMask で署名し，レジストリの対応する関数を呼び出すために送信する．

管理者用の画面では，デプロイ申請や停止提案を確認し，承認または棄却を選択できる．承認した場合は，対応するデプロイまたは停止の Tx を作成し，送信する．接続中のウォレットアドレスがレジストリに登録された管理者アドレスと一致する場合にのみ，管理者用の操作項目を表示する．ただし，これはフロントエンド上の表示制御であり，操作権限の強制はレジストリのアクセス制御が行う．

6. 評価

提案システムを，静的解析，機能評価，および処理量の計測の 3 点から評価する．機能評価および処理時間の計測は，5 章で構築した GoQuorum のテストネットワーク上で行った．ただし，停止の有効性に関するユニットテストとガス消費量の計測には，ローカル EVM として Hardhat 2.28.6 の Hardhat Network を用いた．

6.1 コントラクトの静的解析

実装したレジストリ，プロキシ，およびロジックコントラクトを対象に，静的解析ツール Slither 0.11.5 [16] を適用した．検出された各指摘が，3 章の脅威モデルの下で 4 つの性質を破る経路となるかを確認した．検出 34 件の内訳と詳細を付録 A.1 に示す．

重大度 High の `controlled-delegatecall` については，委譲先が管理者の監査・承認を経たロジックコントラクトに限定され，想定する攻撃者はその値を指定できない．その他の指摘を含めて確認した結果，脅威モデルの下で 4 つの守るべき性質を破る経路は確認されなかった．

表 2 守るべき性質ごとの試験操作と結果

性質	試験した操作	結果
性質 1	管理者・参加者・未登録のアカウントからの直接デプロイ Tx	ノードが Tx 受理を拒否
性質 2	参加者のアカウントからレジストリの管理者用関数の実行	すべて `revert`
性質 3	停止中のプロキシへの呼び出し	読み取りを含めすべて `revert`
性質 4	参加者のアカウントからプロキシへ直接のアップグレード要求	委譲先アドレスが変化せず

6.2 機能評価

3 章に示した 4 つの性質に対して，それぞれの性質に反する操作を試行した．操作と結果を表 2 に示す．

性質 1 の管理外コントラクトの追加防止について，デプロイ権限を剥奪した後の管理者，参加者，および未登録のアカウントから，レジストリを介さないコントラクトのデプロイ Tx を送信した．いずれの Tx もノードによって受理を拒否され，ブロックに取り込まれなかった．ここで確認したのは，外部アカウントからの直接デプロイの拒否であり，コントラクト内部からの生成経路については，4 章で述べた CREATE および CREATE2 の有無に関する管理者の監査に依存する．

性質 2 のライフサイクル操作の権限制御について，参加者のアカウントからレジストリのデプロイ，アップグレード，および停止の各関数の呼び出しを試行した結果，いずれもレジストリのアクセス制御により `revert` された．性質 3 の停止の有効性について，停止中のプロキシコントラクトに対して，状態変更を伴う呼び出しと読み取り専用の呼び出しを試みたところ，いずれも `revert` された．性質 4 のアップグレード経路の限定について，参加者のアカウントからプロキシコントラクトに対して直接ロジックコントラクトの差し替えを要求した．その結果，ERC-1967 のストレージスロットに保持されたロジックコントラクトのアドレスは変化しなかった．

また，監査中，稼働中，停止判断中，停止中の各状態間の遷移が想定した条件に従って行われることを確認した．アップグレード時には，プロキシコントラクトのアドレスとストレージを維持したまま，参照するロジックコントラクトが変更された．

© 2026 Information Processing Society of Japan

表 3　デプロイ経路によるガス消費量の比較

コード長 [B]	直接 [gas]	レジストリ経由 [gas]	増加量 [gas]
1,000	716,139	1,100,230	384,091
10,000	2,554,030	2,945,612	391,582
24,000	5,413,569	5,820,142	406,573

6.3　管理に伴う処理量の増加

EVM 上での Tx 実行に伴う計算資源の消費量は，ガス消費量として表される．本テストネットワークではガス価格を 0 に設定しているため金銭的な費用は生じないが，管理機構による処理量の増加を比較する指標としてガス消費量を用いる．ガス消費量は実行される EVM の命令コストやストレージの状態等に依存し，同一の実行条件下では再現可能な指標である．計測はローカル EVM 上で行い，各 Tx のレシートに含まれる `gasUsed` を参照した．参考として，Ether 送金でも 2.1 万 gas を要する [17].

バイトコード長が 1,000，10,000，24,000 バイトの 3 種類のテスト用コントラクトを対象とした．24,000 バイトは，EIP-170 で定められたコントラクトコードサイズの上限 24,576 バイトを考慮して設定した．レジストリを経由してデプロイする場合と，レジストリを経由せず同一のロジックコントラクトとプロキシコントラクトを直接デプロイする場合のガス消費量を計測した．結果を表 3 に示す．バイトコード長の増加に伴ってデプロイ全体のガス消費量は増加する一方，レジストリを経由することによるガス消費量の増加量は約 38 万から 41 万 gas の範囲であり，大きな変化は見られなかった．また，テストネットワーク上で各経路について 10 回ずつデプロイを行った結果，平均所要時間はいずれも約 10 秒であった．

コントラクト利用時の追加コストを確認するため，状態変数を 1 加算するカウンター関数を対象に，ロジックコントラクトを直接呼び出す場合と，プロキシコントラクトを経由して呼び出す場合のガス消費量を比較した．その結果，プロキシコントラクトを経由する場合は，直接呼び出す場合と比べて，3.9 万 gas 増加した．

7.　考察

静的解析の指摘については，脅威モデルの下で 4 つの性質を破る具体的な経路は確認されなかった．本実装のプロキシコントラクトでは，処理の委譲に `delegatecall` やアセンブリを用いるため，これらが静的解析の指摘対象となった．したがって，検出結果の重大度だけでなく，実装上の利用目的と攻撃者が操作可能な範囲を踏まえて判断する必要がある．

機能評価では，試験したアカウントと操作の範囲で，外部アカウントからレジストリを介さずに行う直接デプロイが拒否されることを確認した．これに加えて，4 章で述べたコントラクト生成機能に対する監査を前提とすることで，3 章の信頼前提の下では，ネットワーク構築後に管理外のコントラクトが追加される経路を排除できる．

本提案では，デプロイ，アップグレード，および停止の実行権限を管理者に集中させている．これにより責任の所在が明確になる一方，管理者への信頼と運用負荷が集中し，管理者の不在や障害によってライフサイクル操作を実行できなくなる可能性がある．この課題に対しては，TRUST [10] のように，承認権限を複数の主体に分散する構成への拡張が考えられる．

性質 1 を満たすためには，管理者がコントラクト生成機能を監査で確実に検出する必要がある．CREATE および CREATE2 命令は，デプロイ用バイトコードの検査によって機械的に確認できるため，レジストリの処理の一部として自動化することで監査負荷の軽減が期待できる．ただし `delegatecall` 先のコードは実行時に決定される場合があるため，バイトコードの走査だけでは生成経路を網羅できない．また，監査では生成機能の有無だけでなく，コントラクトに脆弱性がないかを確認する必要がある．これらの監査は新規デプロイやアップグレードのたびに必要となり，管理対象の増加に伴って管理者の負荷も増加する．

デプロイ時の処理増加は，デプロイ申請や承認に伴うレジストリへの記録，バイトコードのハッシュ計算などから生じる．バイトコード長が 24 倍になるのに対して，追加のガス消費量の差は約 2.2 万 gas にとどまったことから，計測した範囲では固定的な管理処理の占める割合が支配的であった．呼び出し時の処理増加は，ロジックコントラクトへの委譲，レジストリへの停止状態の問い合わせ，および利用記録から生じる．3.9 万 gas のうち，委譲と停止判定が約 2.1 万 gas，利用記録が約 1.8 万 gas を占めた．これらは呼び出しごとに固定回数実行される処理であり，個々のコントラクトの規模には依存しないと考えられる．

デプロイの所要時間に経路による顕著な差が出なかったのは，本実験環境では，ガス消費量の差よりもブロック生成間隔の影響が大きかったためと考えられる．本実装では，直接デプロイではロジックコントラクトとプロキシコントラクトのデプロイ，レジストリ経由では申請と承認を順に実行するため，いずれも 2 件の Tx を要する．ブロック生成間隔を 5 秒とした本実験環境では，平均所要時間はいずれも約 10 秒となった．GoQuorum では 1 ブロック当たりのガス上限を設定できるため，想定する負荷に応じて設定値を調整する必要がある．

本提案の実装には GoQuorum を用いたが，公式リポジトリ [6] は 2026 年 6 月にアーカイブされている．基盤に要求するのは，3 章に示したとおり，アカウント単位で Tx の送信可否およびコントラクトのデプロイ可否を制御できることである．そのため，同等の権限制御機能を備える EVM 互換基盤であれば，本提案の構成を適用可能であると考えられる．

© 2026 Information Processing Society of Japan

8. おわりに

スマートコントラクトの不変性と保守性の両立という課題に対し，コンソーシアム型ブロックチェーンを対象に，基盤の権限管理によるデプロイ経路の限定と，レジストリによるライフサイクル管理を組み合わせた管理システムを提案した．また，GoQuorum 上に実装して静的解析，機能評価，および処理量の計測により評価した．

デプロイ経路の限定については，デプロイ権限を剥奪した管理者，参加者，および未登録のアカウントから送信したレジストリを介さないデプロイ Tx が，いずれもノードによって拒否されることを確認した．ライフサイクル管理については，各状態遷移が管理者の承認を経て実行され，管理者以外による操作はレジストリのアクセス制御によって拒否されることを確認した．実装に対する静的解析では，脅威モデルの下で 4 つの性質を破る経路は確認されなかった．管理に伴う処理量の増加については，デプロイ時の追加ガス消費量は，バイトコード長による変化が小さく，呼び出し時の追加ガス消費量は委譲，状態の問い合わせ，および利用の記録から生じることを示した．

今後の課題として，コントラクト生成経路の検査の自動化，承認権限の複数組織への分散，および管理対象数や Tx 頻度を増加させた環境での性能評価が挙げられる．また，GoQuorum 以外の EVM 互換基盤への移植を通じて，本提案の適用可能性を検証する必要がある．

参考文献

[1] Zheng, Z., Xie, S., Dai, H., Chen, X. and Wang, H.: An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends, *2017 IEEE International Congress on Big Data (BigData Congress)*, pp. 557–564 (online), DOI: 10.1109/BigDataCongress.2017.85 (2017).

[2] Buterin, V.: Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform, `https://ethereum.org/en/whitepaper/` (2014). (accessed 2026-08-05).

[3] Chen, X., He, S., Sun, L., Zheng, Y. and Wu, C. Q.: A Survey of Consortium Blockchain and Its Applications, *Cryptography*, Vol. 8, No. 2 (online), DOI: 10.3390/cryptography8020012 (2024).

[4] Qasse, I., Ali, I. M., Ahmed, N., Hamdaqa, M. and Jónsson, B. T.: The Myth of Immutability: A Multivocal Review on Smart Contract Upgradeability, arXiv:2504.02719, `https://arxiv.org/abs/2504.02719` (2025). (accessed 2026-08-13).

[5] Atzei, N., Bartoletti, M. and Cimoli, T.: A Survey of Attacks on Ethereum Smart Contracts (SoK), *Principles of Security and Trust* (Maffei, M. and Ryan, M., eds.), Berlin, Heidelberg, Springer Berlin Heidelberg, pp. 164–186 (2017).

[6] ConsenSys: GoQuorum: A Permissioned Implementation of Ethereum Supporting Data Privacy, GitHub, `https://github.com/Consensys/quorum`. (accessed 2026-08-05).

[7] Barros, G. and Gallagher, P.: EIP-1822: Universal Upgradeable Proxy Standard (UUPS), Ethereum Improvement Proposals, `https://eips.ethereum.org/EIPS/eip-1822` (2019). (accessed 2026-07-28).

[8] Wöhrer, M. and Zdun, U.: Smart contracts: security patterns in the ethereum ecosystem and solidity, *2018 International Workshop on Blockchain Oriented Software Engineering (IWBOSE)*, pp. 2–8 (online), DOI: 10.1109/IWBOSE.2018.8327565 (2018).

[9] Qasse, I., Hamdaqa, M. and Jónsson, B. T.: Immutable in principle, upgradeable by design: exploratory study of smart contract upgradeability, *Empirical Software Engineering*, Vol. 31, No. 2, p. 39 (online), DOI: 10.1007/s10664-025-10779-y (2025).

[10] Cruz, E., Souza, Y., Alcantara, G. and Peixoto, M.: TRUST: A lifecycle-oriented system architecture for governed evolution of smart contract applications, *Information and Software Technology*, Vol. 198, p. 108210 (online), DOI: https://doi.org/10.1016/j.infsof.2026.108210 (2026).

[11] Wang, Y., Ali, J., Arshad, J. and Liu, Y.: A Proxy-Layer Approach to Secure Smart Contract Deployment on Private EVM-Based PoA Blockchains , *2023 IEEE International Conference on Blockchain (Blockchain)*, Los Alamitos, CA, USA, IEEE Computer Society, pp. 109–112 (online), DOI: 10.1109/Blockchain60715.2023.00027 (2023).

[12] Foundation, H.: Hyperledger Fabric Documentation v2.5, `https://hyperledger-fabric.readthedocs.io/en/release-2.5/`. (accessed 2026-08-12).

[13] Buterin, V.: EIP-1014: Skinny CREATE2, Ethereum Improvement Proposals, `https://eips.ethereum.org/EIPS/eip-1014` (2018). (accessed 2026-08-20).

[14] ConsenSys: Permissioning, GoQuorum Documentation, `https://github.com/Consensys/doc.goquorum/`. (accessed 2026-08-05).

[15] Palladino, S., Giordano, F. and Croubois, H.: ERC-1967: Proxy Storage Slots, Ethereum Improvement Proposals, `https://eips.ethereum.org/EIPS/eip-1967` (2019). (accessed 2026-08-15).

[16] Feist, J., Grieco, G. and Groce, A.: Slither: A Static Analysis Framework for Smart Contracts, *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*, pp. 8–15 (online), DOI: 10.1109/WETSEB.2019.00008 (2019).

[17] Wood, G.: Ethereum: A Secure Decentralised Generalised Transaction Ledger, Ethereum Project Yellow Paper, `https://ethereum.github.io/yellowpaper/paper.pdf`. (accessed 2026-08-01).

付　　録

A.1 静的解析の検出結果

6 章で述べた Slither による検出 34 件について，検出器ごとの内訳を表 A·1 に示す．

重大度が High の `controlled-delegatecall` は，プロキシコントラクトがロジックコントラクトの初期化関数を `delegatecall` で呼び出す箇所に対する検出である．Slither は委譲先アドレスが変数で与えられる構文に対して，宛先が外部から指定されうるものとして検出する．本実装では，委譲先アドレスは管理者の監査・承認を経てデ

© 2026 Information Processing Society of Japan

表 A·1 Slither の検出結果

重大度	検出器	件数
High	`controlled-delegatecall`	2
Medium	`incorrect-equality`	2
Low	`timestamp`	8
	`reentrancy-benign`	1
Info.	`assembly`	15
	`low-level-calls`	4
	`naming-convention`	2

プロイされたロジックコントラクトのアドレスであり，3 章で想定する攻撃者はその値を指定できない．また，初期化関数には一度のみ実行可能とする制御を設けている．したがって，想定する攻撃者がこの `delegatecall` を利用して，プロキシコントラクトの状態を任意に変更することはできない．

重大度が Medium の `incorrect-equality` は，コントラクトの状態を表す列挙型に対する等価比較の検出である．Slither は，残高やタイムスタンプなど外部から影響を受ける値に対する厳密な等価比較を潜在的な問題として検出する．一方，本実装で検出された比較の対象は，レジストリが管理するライフサイクル状態を表す値であり，外部から任意に操作される値ではない．

重大度が Low および Informational の検出は，プロキシコントラクトで使用するアセンブリおよび低レベル呼び出し，状態更新を伴う外部呼び出し，記録用途の `block.timestamp`，ならびに命名規則に関するものであった．これらはいずれも，ライフサイクル操作の権限やデプロイ経路，停止判定，アップグレード経路を変更するものではない．

以上より，Slither によって検出された指摘はいずれも，3 章の脅威モデルの下で，4 つの守るべき性質を破る経路とはならないことを確認した．

© 2026 Information Processing Society of Japan